Doc. no.:	P3060R1
Date:	2024-2-14
Audience:	LEWG
Reply-to:	Weile Wei <weilewei09@gmail.com> Zhihao Yuan <zy@miator.net>

Add std::views::upto(n)

Revision History

Since R0

- Move `upto` from the `std::ranges` namespace into `std::views`
- Use a more convincing example
- Incorporate with the current `iota` wording in the standard

Abstract

We propose adding `std::views::upto(n)` to the C++ Standard Library as a range adaptor that generates a sequence of integers from `0` to `n-1`.

Motivation

Currently, `iota(0, ranges::size(rng))` does not compile due to mismatched types (see point 1 in Background section). Then, users need to write a workaround like `iota(range_size_t<decltype(rng)>{}, ranges::size(rng))`, which is not straightforward and cumbersome. An example illustrating this issue is available at Compiler Explorer x8nWxqE9v:

C++23	`std::vector rng(5, 0);`
	`// auto res1 = views::iota(0, ranges::size(rng)); // does not compile`
	`auto res2 = iota(range_size_t<decltype(rng)>{}, ranges::size(rng));`
	`std::print("{}", res2); // [0, 1, 2, 3, 4]`

P3060	`std::vector rng(5, 0);` `std::print("{}", views::upto(ranges::size(rng))); // [0, 1, 2, 3, 4]`

`std::views::upto(n)` eases this pattern by providing a straightforward method to generate integer sequences, improving readability and killing arcane code.

Implementation and Usage

Implementation details:

```
namespace std::views {
    inline constexpr auto upto = [] <std::integral I> (I n) {
        return std::views::iota(I{}, n);
    };
}
```

Usage:

```
int main() {
    std::vector rng(5, 0);
    std::print("{}", views::upto(ranges::size(rng))); // [0, 1, 2, 3, 4]
}
```

The implementation is confirmed to work with clang version 16.0.0+, gcc version 11.1+, and msvc v19.32+: Compiler Explorer 4MKh317dr

Background

Two preceding proposals have provided fundation for `std::views::upto(n)`:

1. *P2214R2: A Plan for C++26 Ranges*[1] highlights the issue with `views::iota(0, r.size())` not compiling due to mismatched types. `std::ranges::views::iota` requires both arguments to be of the same type, or at least commonly comparable. This becomes problematic when comparing `int` (often 32-bit) with `std::size_t` (often 64-bit), which is usually wider on 64-bit systems. The same example from above Motivation section can be viewed at Compiler Explorer x8nWxqE9v.

```
std::vector rng(5, 0);
auto res1 = iota(0, ranges::size(rng)); // does not compile
```

2. *P1894R0: Proposal of std::upto, std::indices and std::enumerate*[2] proposed an implementation (see below) that our proposal refines. Our approach offers a more consistent interface, fitting seamlessly with existing standard library.

```
// std::upto implementation example
// P1894R0: Proposal of std::upto, std::indices and std::enumerate
namespace std {
    template<typename Int>
    constexpr auto upto(Int&& i) noexcept {
        return std::ranges::iota_view{Int(),std::forward<Int>(i)};
    }
}
```

Technical Decisions

1. Limiting to `std::integral`: This constraint ensures functionality is restricted to integral types, yielding predictable behavior and avoiding the pitfalls of generic types. A more relaxed constraint to `std::default_initializable` and `std::incrementable` would allow iterators to compile but might introduce undefined behavior. An example illustrating this issue is available at Compiler Explorer a3neb43a4:

```cpp
namespace std::views {
    // a more relaxed design pattern, compiled but UB
    inline constexpr auto upto2 =
        []<typename T> (T n)
            requires std::default_initializable<T> && std::incrementable<T>
        {
            return std::views::iota(T{}, n);
        };
}

int main() {
    int myint = 5;
    int* ptr = &myint;

    // !!!compiled but undefined behavior!!!
    auto up_to_five = std::views::upto2(CustomIterator{ptr});
    for (auto i : up_to_five) {
        std::cout << *i << ' ';
    }

    return 0;
}
```

2. Lambda-based Approach: Using a lambda is consistent with the established range adaptor patterns. Moreover, the use of `constexpr` allows for the evaluation to occur at compile-time.

3. Leveraging Existing `iota_view` Instead of Creating a New `upto_view`: Introducing `upto_view` would mean adding a component similar to what already exists, causing confusion and maintainability issues for users. By leveraging `iota_view`, we simplify the implementation and reuse of what the current C++ Standard Library offers. Additionally, any future optimizations to `iota_view` will automatically benefit `std::views::upto`. Therefore, by extending `iota_view`, `std::views::upto` becomes more maintainable, efficient, and simple.

Wording

Add a new parapgrah under [range.iota.overview]:

> 26.6.4.1 Overview [range.iota.overview]
>
> The name `views::upto` denotes a customization point object ([customization.point.object]). Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `T` does not model `integral`, then `views::upto(E)` is ill-formed. Otherwise, the expression `views::upto(E)` is expression-equivalent to `views::iota(T(), E)`.

References

1. P2214R2 *A Plan for C++26 Ranges.* https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r0.html ↩

2. P1894R0 *Proposal of std::upto, std::indices and std::enumerate.* https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2019/p1894r0.pdf ↩